CORRIGÉ · PROBLÈME · ANNALES SPÉ

Flots de données

Énoncé

1. Flots de données paresseux

OCaml exclusivement. `let list_finite = [1; 2; 3]` et `let rec list_ones = 1 :: list_ones`.

1. Croquis en maillons de `list_finite` et `list_ones` en mémoire ; quelle région de la mémoire ?

Le code suivant provoque `Stack overflow during evaluation (looping recursion?)` :

```
let list_fibo : int list =
  let rec list_fibo_with_internal_state (a:int) (b:int) : int list =
    a :: (list_fibo_with_internal_state b (a+b))
  in
  list_fibo_with_internal_state 0 1
```

1. Traduire *stack* ; expliquer l'erreur.
2. Avec `ocamlopt fibo.ml -o fibo` puis `./fibo` : l'une des étapes échoue-t-elle ?

`type 'a stream = Nil | Cons of 'a * (unit -> 'a stream)`. Exemples : `let rec ones = Cons (1, fun () -> ones)` ; `fibo` défini par `fibo_with_internal_state a b = Cons (a, fun () -> fibo_with_internal_state b (a+b))`.

1. Constante `integers : int stream` (entiers naturels croissants).
2. `range (a:int) (b:int) : int stream` (entiers de a inclus à b exclu).
3. Toute suite (σ_n) de chaînes est-elle codée par un `string stream` ?
4. `hd (u:'a stream) : 'a`, avec `Failure "hd"` si `u` est vide.
5. `tl (u:'a stream) : 'a stream`, avec `Failure "tl"`.
6. `of_list (l:'a list) : 'a stream`.
7. `iter (f:'a -> unit) (t:int) (u:'a stream) : unit` : applique `f` aux `t` premières valeurs existantes.
8. `map (f:'a -> 'b) (u:'a stream) : 'b stream`.
9. `zip (w:'a stream array) : 'a array stream` : flot des tableaux $[u_n, v_n, \dots]$, vide dès que l'un des flots l'est.

10. `intertwin (u:'a stream) (v:'a stream) : 'a stream` : alterne les éléments de `u` et `v`, en continuant avec l'autre quand l'un est épuisé.

```
let rec product_wrong u1 u2 =
  match u1, u2 with
  | _, Nil | Nil, _ -> Nil
  | Cons (hd1, tl_begetter1), Cons (hd2, tl_begetter2) ->
      let part1 = product_wrong (tl_begetter1 ()) u2 in
      let part2 = map (fun y -> (hd1, y)) (tl_begetter2 ()) in
      Cons ((hd1, hd2), fun () -> intertwin part1 part2)
```

1. Montrer que `product_wrong u1 u2` ne termine pas toujours, et qu'il suffit de déplacer quelques mots pour le corriger.
2. Signification du `0` dans `return 0` (C) ou `exit 0` (OCaml).
3. Énoncer le théorème de Turing sur l'arrêt.
4. On admet une fonction `run : string -> int -> int` qui termine toujours et renvoie 0 si le code x termine sans erreur en moins de n étapes, 1 sinon. Construire ou réfuter l'existence de `filter (f:'a -> bool) (u:'a stream) : 'a stream` (éléments de `u` vérifiant `f`, dans l'ordre ; par exemple `filter is_negative ones` vaut `Nil`).

2. Estimer le cardinal d'un flot : HyperLogLog

Cardinal d'un flot = nombre de valeurs distinctes. Le premier bit activé $\varrho(h)$ d'un entier $h \in [\![0, 2^{62}-1]\!]$ est le plus grand i tel que les $i-1$ bits de droite sont nuls ; $\varrho(0) = 63$ (exemples : $\varrho(1) = 1$, $\varrho(6) = 2$, $\varrho(8) = 4$). Hachage par `String.hash`.

1. Écrire `ffs (h:int) : int`.

Deux algorithmes sur les t premiers termes : `cardinality_nf` remplit une table de hachage et renvoie son nombre d'éléments ; `cardinality_fm` garde $r = \max \varrho(H(u))$ et renvoie 2^r.

1. Comparer leurs complexités en espace.
2. Écrire `harmonic_mean (z:float array) : float` ($\text{Harm} = m(\sum \frac{1}{z_j})^{-1}$).

HyperLogLog ($m = 16$, $\alpha_{16} = 0{,}6731$) : le terme u va dans le sous-flot $i = H(u) \bmod m$, et $r_i = \max \varrho(H(u)/m)$ sur ce sous-flot ; estimation $s \simeq \alpha_m \cdot m \cdot \text{Harm}(2^{r_0}, \dots, 2^{r_{m-1}})$.

1. Écrire `hll_update (r:int array) (x:string) : unit` (met à jour r avec le nouveau terme x).
2. Écrire `mean_estimate (r:int array) : float`.
3. Dans le code multi-fils donné (deux fils remplissent `r_u` et `r_v` par `hll_update` ; le fil principal calcule toutes les 5 secondes `Array.init m (fun i -> max r_u.(i) r_v.(i))` et affiche l'estimation), identifier les courses critiques et proposer un mécanisme de correction.

3. Chiffrement par flot de booléens

$X(\varphi,\psi) = (\varphi \vee \psi) \wedge (\neg\varphi \vee \neg\psi)$; règles de déduction naturelle en annexe.

1. Arbres de preuve de $X(\varphi,\varphi) \vdash \bot$ et $\bot \vdash X(\varphi,\varphi)$.
2. Arbres de preuve de $X(\bot,\psi) \vdash \psi$ et $\psi \vdash X(\bot,\psi)$.
3. $\oplus(x,y)$ désigne l'évaluation de $X(x,y)$. Justifier que $(x,y) \mapsto \oplus(\oplus(x,x),y)$ et $(x,y) \mapsto y$ sont égales.

LFSR de longueur m, germe $(b_i)_{0\leqslant i<m}$, connexion $C \subseteq [\![0, m-1]\!]$: $b_{n+m} = \bigoplus_{i\in C} b_{n+i}$ ($\oplus$ est associative et commutative ; somme vide $= F$). Germe : `bool array` ; connexion : `int list`.

1. Écrire `lfsr (g:bool array) (c:int list) : bool stream`.
2. Avec `type formula = Var of int | Not of formula | And of formula * formula | Or of formula * formula`, écrire `logical_map (phi:formula) (w:bool stream array) : bool stream` : le flot des évaluations $\varphi(b_n^{(1)}, \dots, b_n^{(k)})$ (φ ne dépend que de $x_1, \dots, x_k$, `w` de longueur k).
3. Alice et Bob partagent k LFSR et une formule ψ_0. Alice envoie publiquement $X(\psi_0(b_n^{(1)}, \dots, b_n^{(k)}), c_n)$. Bob peut-il retrouver (c_n) ?

4. Mots de Dyck et algorithme de Heap

`let rec dyck = let f (x,y) = "a" ^ x ^ "b" ^ y in Cons ("", fun () -> map f (product dyck dyck))` ; D est l'ensemble des mots de ce flot.

1. Donner une grammaire non contextuelle G avec $L(G) = D$.
2. Énoncer une propriété de G qui assure que `dyck` n'énumère jamais deux fois le même mot, et la démontrer.

Algorithme de Heap : Heap(k, t) : si $k = 1$, afficher t ; sinon, pour $i = 0$ à $k-1$: exécuter Heap($k-1$, t), puis si k est pair échanger $t[i]$ et $t[k-1]$, sinon échanger $t[0]$ et $t[k-1]$.

1. Nombre $A(k)$ d'affichages lors de Heap(k, t).
2. Montrer : après Heap(k, t), si k est impair, t est inchangé ; si k est pair, les k premières cases ont subi une permutation circulaire vers la droite.
3. Montrer que Heap(k, t) affiche chaque permutation des k premières cases (les autres inchangées).
4. Écrire `pivot (k:int) (i:int) (t:'a array) : unit` (lignes 6 à 9).
5. Écrire `permstream_of_array (t:'a array) : 'a array stream`, flot fini des permutations dans l'ordre de l'algorithme de Heap, économe en mémoire.

Décoder l'énoncé

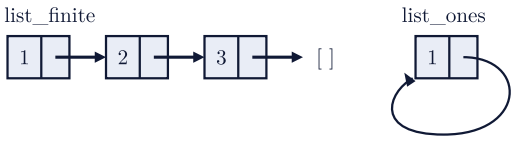
Ce que l'énoncé te donne.

- OCaml est strict : les arguments d'une fonction sont évalués avant l'appel.
- `fun () -> e` retarde le calcul de `e`.
- $\varrho(h)$ est la position (à partir de 1) du bit 1 le plus à droite.
- $X(\varphi, \psi)$ est le « ou exclusif » de φ et ψ.
- `dyck` = le mot vide, puis tous les $a\,x\,b\,y$ avec x, y dans `dyck`.

Ce qu'on te demande. Comprendre pourquoi certaines définitions récursives bouclent, et programmer avec des flots paresseux.

Corrigé

Question 1



chaque maillon : (tête, pointeur vers la queue) ; stockés dans le tas

`list_finite` : trois maillons terminés par la liste vide. `list_ones` : un seul maillon dont la queue pointe sur lui-même (liste cyclique).

`list_finite` est une chaîne de trois maillons, le dernier pointant vers la liste vide. `list_ones` est un **seul** maillon dont la queue pointe sur lui-même : une liste cyclique, donc « infinie » quand on la parcourt.

Ces blocs sont stockés dans le **tas** (zone de mémoire gérée par le ramasse-miettes d'OCaml), pas dans la pile.

Question 2

Stack se traduit par **pile** (la pile d'appels).

Déroulement : pour évaluer `list_fibo_with_internal_state 0 1`, il faut construire `0 :: (...)`, donc d'abord évaluer la queue `list_fibo_with_internal_state 1 1` (OCaml est strict : on évalue les arguments d'un constructeur avant de le construire). Celle-ci demande `list_fibo_with_internal_state 1 2`, puis `2 3`, `3 5`… Aucun appel ne se termine, et chacun empile un bloc d'activation : la pile déborde.

Pourquoi `list_ones` marche : `let rec list_ones = 1 :: list_ones` est une définition récursive

de **valeur** (un constructeur appliqué au nom défini), qu'OCaml traite en créant directement le maillon cyclique, sans appel de fonction.

Question 3

La **compilation réussit** : le code est syntaxiquement correct et bien typé (`list_fibo : int list`). L'erreur se produit à l'**exécution** de `./fibo` : la récursion infinie épuise la pile et le programme s'arrête avec l'exception `Stack_overflow` (ou une erreur de segmentation, selon le système).

Question 4

```
let integers : int stream =
  let rec from n = Cons (n, fun () -> from (n + 1)) in
  from 0
```

L'appel `from (n + 1)` est retardé par `fun () ->` : la définition termine immédiatement.

Question 5

```
let rec range (a:int) (b:int) : int stream =
  if a >= b then Nil
  else Cons (a, fun () -> range (a + 1) b)
```

Question 6

Non, pas forcément. Un flot est défini par un programme OCaml, c'est-à-dire un texte fini : il y a un ensemble **dénombrable** de tels programmes. L'ensemble des suites de chaînes n'est pas dénombrable : déjà les suites à valeurs dans $\{\texttt{"0"}, \texttt{"1"}\}$ sont en bijection avec $\{0,1\}^{\mathbb{N}}$, non dénombrable (argument diagonal de Cantor). Il existe donc des suites qu'aucun flot ne code.

Quand c'est possible : si la suite est **calculable**, c'est-à-dire s'il existe une fonction `sigma : int -> string` qui calcule σ_n, alors

```
let stream_of_sequence (sigma : int -> string) : string stream =
  let rec from n = Cons (sigma n, fun () -> from (n + 1)) in
  from 0
```

Question 7

```
let hd (u : 'a stream) : 'a =
  match u with
  | Nil -> failwith "hd"
  | Cons (x, _) -> x
```

Question 8

```
let tl (u : 'a stream) : 'a stream =
  match u with
  | Nil -> failwith "tl"
  | Cons (_, f) -> f ()
```

C'est l'appel `f ()` qui calcule effectivement la suite du flot.

Question 9

```
let rec of_list (l : 'a list) : 'a stream =
  match l with
  | [] -> Nil
  | x :: q -> Cons (x, fun () -> of_list q)
```

Question 10

```
let rec iter (f : 'a -> unit) (t : int) (u : 'a stream) : unit =
  if t > 0 then
    match u with
    | Nil -> ()                          (* moins de t valeurs existantes *)
    | Cons (x, g) -> f x; iter f (t - 1) (g ())
```

On ne force la queue `g ()` que si l'on en a besoin. Si $t \leqslant 0$, rien ne se passe. La récursion est terminale : pas de risque de débordement de pile.

Question 11

```
let rec map (f : 'a -> 'b) (u : 'a stream) : 'b stream =
  match u with
  | Nil -> Nil
  | Cons (x, g) -> Cons (f x, fun () -> map f (g ()))
```

Question 12

```
let is_nil u = match u with Nil -> true | Cons _ -> false

let rec zip (w : 'a stream array) : 'a array stream =
  if Array.exists is_nil w then Nil
  else Cons (Array.map hd w, fun () -> zip (Array.map tl w))
```

On teste le vide par filtrage (`is_nil`) plutôt que par `u = Nil` : l'égalité structurelle pourrait tenter de comparer des fonctions. Les queues ne sont calculées (`Array.map tl w`) que lorsque l'on demande la suite.

Question 13

```
let rec intertwin (u : 'a stream) (v : 'a stream) : 'a stream =
  match u with
  | Nil -> v                                    (* u épuisé : on continue avec v *)
  | Cons (x, g) -> Cons (x, fun () -> intertwin v (g ()))   (* on alterne *)
```

Vérification sur `intertwin (range 0 2) fibo` : 0, puis F_0 (rôles échangés), puis 1, puis F_1 ; `range` est alors épuisé et l'on continue avec $F_2, F_3, \ldots$: c'est l'exemple de l'énoncé.

Question 14

Non-terminaison : `part1 = product_wrong (tl_begetter1 ()) u2` est calculé **avant** de renvoyer le `Cons`. Cet appel calcule à son tour le `part1` suivant, et ainsi de suite le long de tout `u1`. Si `u1` est infini (par exemple `ones`) et `u2` non vide, la récursion ne s'arrête jamais.

Correction : il suffit de déplacer `fun () ->` pour que `part1` et `part2` ne soient calculés qu'à la demande :

```
let rec product u1 u2 =
  match u1, u2 with
  | _, Nil | Nil, _ -> Nil
  | Cons (hd1, tl_begetter1), Cons (hd2, tl_begetter2) ->
      Cons ((hd1, hd2), fun () ->
        let part1 = product (tl_begetter1 ()) u2 in
        let part2 = map (fun y -> (hd1, y)) (tl_begetter2 ()) in
        intertwin part1 part2)
```

Chaque appel renvoie maintenant un `Cons` immédiatement. Chaque couple (u_i, v_j) apparaît : les couples avec u_0 en tête sont dans `part2` ou en tête, les autres dans `part1`, et `intertwin` alterne entre les deux, donc tout élément d'un des deux flots finit par sortir.

Question 15

C'est le **code de retour** du processus, transmis au système d'exploitation (ou au processus parent, par exemple le terminal). Par convention, 0 signifie que le programme s'est terminé **sans erreur** ; une valeur non nulle signale un échec.

Question 16

> Il n'existe aucun programme qui, recevant le code d'un programme P et une entrée x, termine toujours et répond correctement à la question « l'exécution de P sur x termine-t-elle ? ». Le problème de l'arrêt est **indécidable**.

Question 17

On réfute l'existence d'une telle fonction `filter` (qui termine toujours, puisque `filter is_negative ones` doit valoir `Nil`).

Supposons qu'elle existe. Pour un code source x, on définit le flot

```
let witness (x : string) : int stream = map (run x) integers
(* terme n : 0 si x termine en moins de n étapes, 1 sinon *)

let halts (x : string) : bool =
  match filter (fun v -> v = 0) (witness x) with
  | Nil -> false        (* aucun 0 : x ne termine jamais *)
  | Cons _ -> true      (* un 0 existe : x termine *)
```

`run` termine toujours, donc `witness x` est un flot infini bien défini. Le flot filtré est vide si et seulement si aucun terme ne vaut 0, c'est-à-dire si x ne termine pas. Comme `filter` termine, `halts` termine toujours et décide l'arrêt, ce qui contredit le théorème de Turing.

Remarque : on peut écrire un filtre **partiel**, qui cherche le prochain élément valide et boucle s'il n'y en a plus (`filter is_negative ones` ne terminerait alors pas) ; c'est la spécification « vaut `Nil` » qui est impossible.

Question 18

```
let ffs (h : int) : int =
  if h = 0 then 63                          (* convention *)
  else begin
    let rec aux h i = if h land 1 = 1 then i else aux (h lsr 1) (i + 1) in
    aux h 1
  end
```

Vérification : $6 = 110_{(2)}$: le bit de poids 1 est nul, le suivant vaut 1, donc $\varrho(6) = 2$.

Question 19

- `cardinality_nf` stocke toutes les chaînes distinctes dans la table : $O(s)$ chaînes, donc une mémoire proportionnelle au cardinal (multipliée par la longueur des chaînes).
- `cardinality_fm` ne garde qu'un entier $r \leqslant 63$: mémoire $O(1)$ (6 bits suffisent).

Le second est donc bien plus économe, au prix d'une estimation approchée (et grossière, puisqu'elle ne prend que des puissances de 2).

Question 20

```
let harmonic_mean (z : float array) : float =
  let m = Array.length z in
  let s = Array.fold_left (fun acc x -> acc +. 1. /. x) 0. z in
  float_of_int m /. s
```

Question 21

```
let hll_update (r : int array) (x : string) : unit =
  let h = String.hash x in
  let i = h mod m in                          (* numéro du sous-flot *)
  r.(i) <- max r.(i) (ffs (h / m))            (* nouveau maximum de rho sur ce sous-flot *)
```

Seule la case i change : elle passe de $\max_{u \in U_{t-1}^i}$ à $\max_{u \in U_t^i}$, comme demandé.

Question 22

```
let mean_estimate (r : int array) : float =
  let z = Array.map (fun ri -> 2. ** float_of_int ri) r in   (* les 2^{r_i} *)
  alpha16 *. float_of_int m *. harmonic_mean z
```

On calcule 2^{r_i} en flottant (`1 lsl 63` déborderait pour $r_i = 63$).

Question 23

Accès concurrents : le fil `f1` écrit dans `r_u` (par `hll_update`, qui lit `r.(i)` puis l'écrit), pendant que le fil principal lit `r_u` à la ligne 46 ; de même pour `f2` et `r_v`. Les portions critiques sont donc les appels `hll_update r_u` (ligne 38) et `hll_update r_v` (ligne 42), et la construction de `r` (ligne 46). Le fil principal peut lire un tableau en cours de mise à jour et afficher une estimation incohérente.

Correction : un verrou par tableau (`Mutex`). On encadre chaque mise à jour et la lecture :

```
let lock_u = Mutex.create () and lock_v = Mutex.create () in
let f1 () = iter (fun x -> Mutex.lock lock_u; hll_update r_u x; Mutex.unlock lock_u)
              Int.max_int u in
(* idem pour f2 avec lock_v *)
...
Mutex.lock lock_u; Mutex.lock lock_v;              (* toujours dans le même ordre *)
let r = Array.init m (fun i -> max r_u.(i) r_v.(i)) in
Mutex.unlock lock_v; Mutex.unlock lock_u;
print_float (mean_estimate r)
```

On prend les verrous toujours dans le même ordre pour éviter tout interblocage, et on les relâche avant l'affichage et la pause.

Question 24

Notons $\Delta = \{X(\varphi, \varphi)\}$ et $X = (\varphi \vee \varphi) \wedge (\neg\varphi \vee \neg\varphi)$.

Arbre 1 ($\Delta \vdash \varphi$) : par $\wedge_e$ on obtient $\varphi \vee \varphi$, puis par $\vee_e$ les deux cas donnent φ par l'axiome.

$$\dfrac{\dfrac{\dfrac{}{\Delta \vdash X}\,(\text{ax})}{\Delta \vdash \varphi \vee \varphi}\,(\wedge_e) \qquad \dfrac{}{\Delta, \varphi \vdash \varphi}\,(\text{ax}) \qquad \dfrac{}{\Delta, \varphi \vdash \varphi}\,(\text{ax})}{\Delta \vdash \varphi}\,(\vee_e)$$

Arbre 2 ($\Delta \vdash \neg\varphi$) : le même, avec la seconde composante.

$$\dfrac{\dfrac{\dfrac{}{\Delta \vdash X}\,(\text{ax})}{\Delta \vdash \neg\varphi \vee \neg\varphi}\,(\wedge_e) \qquad \dfrac{}{\Delta, \neg\varphi \vdash \neg\varphi}\,(\text{ax}) \qquad \dfrac{}{\Delta, \neg\varphi \vdash \neg\varphi}\,(\text{ax})}{\Delta \vdash \neg\varphi}\,(\vee_e)$$

Conclusion par $\neg_e$:

$$\dfrac{\dfrac{\vdots}{\Delta \vdash \varphi}\,(\text{arbre 1}) \qquad \dfrac{\vdots}{\Delta \vdash \neg\varphi}\,(\text{arbre 2})}{X(\varphi, \varphi) \vdash \bot}\,(\neg_e)$$

Réciproque : de $\bot$ on tire n'importe quoi :

$$\dfrac{\dfrac{}{\bot \vdash \bot}\,(\text{ax})}{\bot \vdash X(\varphi, \varphi)}\,(\bot_e)$$

Question 25

Notons $\Delta = \{X(\bot, \psi)\}$, avec $X(\bot, \psi) = (\bot \vee \psi) \wedge (\neg\bot \vee \neg\psi)$.

$$\dfrac{\dfrac{\dfrac{}{\Delta \vdash X(\bot, \psi)}\,(\text{ax})}{\Delta \vdash \bot \vee \psi}\,(\wedge_e) \qquad \dfrac{\dfrac{}{\Delta, \bot \vdash \bot}\,(\text{ax})}{\Delta, \bot \vdash \psi}\,(\bot_e) \qquad \dfrac{}{\Delta, \psi \vdash \psi}\,(\text{ax})}{X(\bot, \psi) \vdash \psi}\,(\vee_e)$$

Réciproque $\psi \vdash X(\bot, \psi)$: on construit les deux composantes et on les assemble par $\wedge_i$. Pour la seconde, $\neg\bot$ s'obtient par $\neg_i$ (sous l'hypothèse $\bot$, on a $\bot$ par l'axiome).

$$\dfrac{\dfrac{\dfrac{}{\psi \vdash \psi}\,(\text{ax})}{\psi \vdash \bot \vee \psi}\,(\vee_i) \qquad \dfrac{\dfrac{\dfrac{}{\psi, \bot \vdash \bot}\,(\text{ax})}{\psi \vdash \neg\bot}\,(\neg_i)}{\psi \vdash \neg\bot \vee \neg\psi}\,(\vee_i)}{\psi \vdash X(\bot, \psi)}\,(\wedge_i)$$

Question 26

$\oplus(x, y)$ vaut V si et seulement si exactement un de x, y vaut V : $(x \vee y)$ exige au moins un V, et $(\neg x \vee \neg y)$ au moins un F.

- $\oplus(x, x) = F$ pour tout x (au moins un V et au moins un F parmi x, x est impossible) : c'est la version sémantique de Q24.
- $\oplus(F, y) = y$: si $y = V$, exactement un vaut V ; si $y = F$, aucun. C'est la version sémantique de Q25.

Donc pour tous x, y : $\oplus(\oplus(x, x), y) = \oplus(F, y) = y$. Les deux fonctions sont égales.

x	y	$\oplus(x, x)$	$\oplus(\oplus(x, x), y)$
V	V	F	V
V	F	F	F
F	V	F	V
F	F	F	F

Question 27

```
let lfsr (g : bool array) (c : int list) : bool stream =
  let m = Array.length g in
  (* w contient la fenêtre (b_n, ..., b_{n+m-1}) *)
  let rec from (w : bool array) : bool stream =
    Cons (w.(0), fun () ->
      let next = List.fold_left (fun acc i -> acc <> w.(i)) false c in   (* b_{n+m} *)
      from (Array.init m (fun i -> if i < m - 1 then w.(i + 1) else next)))
  in
  if m = 0 then Nil else from (Array.copy g)
```

Sur les booléens, `a <> b` vaut `true` si et seulement si exactement un des deux est vrai : c'est $\oplus$. Le `fold_left` part de `false` (F, somme vide) et ajoute les b_{n+i} pour $i \in C$. On copie le germe pour ne pas dépendre de modifications ultérieures du tableau passé en argument.

Question 28

```
let rec eval (phi : formula) (b : bool array) : bool =
  match phi with
  | Var j -> b.(j - 1)                (* x_j est la j-ième valeur, indice j-1 *)
  | Not p -> not (eval p b)
  | And (p, q) -> eval p b && eval q b
  | Or (p, q) -> eval p b || eval q b

let logical_map (phi : formula) (w : bool stream array) : bool stream =
  map (eval phi) (zip w)
```

`zip w` fournit, pour chaque n, le tableau $[b_n^{(1)}, \dots, b_n^{(k)}]$; `map` lui applique l'évaluation de φ.

Question 29

Oui, toujours. Bob connaît les germes, les connexions et ψ_0 : il recalcule exactement les mêmes flots $b^{(1)}, \dots, b^{(k)}$ (avec `lfsr`) et le même flot de clé $k_n = \psi_0(b_n^{(1)}, \dots, b_n^{(k)})$ (avec `logical_map`). Il reçoit $e_n = \oplus(k_n, c_n)$ et calcule

$$\oplus(k_n, e_n) = \oplus(k_n, \oplus(k_n, c_n)) = \oplus(\,\oplus\,(k_n, k_n), c_n) = c_n,$$

par associativité de $\oplus$ puis Q26. Un espion qui ne connaît pas la clé ne voit que e_n.

Question 30

$$G : \qquad S \to aSbS \mid \varepsilon.$$

Justification : le flot `dyck` contient le mot vide, puis les mots `"a" ^ x ^ "b" ^ y` pour tous les couples (x, y) de mots du flot (`product` les énumère tous). D est donc le plus petit langage contenant ε et stable par $(x, y) \mapsto axby$: c'est exactement $L(G)$ (récurrence sur la longueur des mots, ou sur la hauteur des arbres de dérivation).

Question 31

Propriété : la grammaire G est **non ambiguë** (chaque mot a un unique arbre de dérivation). Comme `dyck` produit chaque arbre une seule fois (le produit énumère chaque couple une seule fois et f correspond à une règle), elle n'énumère jamais deux fois le même mot.

Lemme : dans un mot $w \in D$, tout préfixe a au moins autant de a que de b, et w a autant de a que de b (par récurrence sur la longueur de la dérivation : c'est vrai pour ε ; pour $axby$, un préfixe de ax a strictement plus de a, axb est équilibré, puis on ajoute un préfixe de y).

Unicité, par récurrence forte sur la longueur. Le mot vide n'a que la dérivation $S \to \varepsilon$. Soit $w \neq \varepsilon$; toute dérivation commence par $S \to aSbS$, et $w = axby$ avec $x, y \in D$. Le préfixe axb est le **plus court préfixe non vide** de w ayant autant de a que de b : d'après le lemme appliqué à x, les préfixes ax' avec x' préfixe de x ont un a de plus que de b. Ce plus court préfixe ne dépend que de w : la découpe (x, y) est donc unique, et par hypothèse de récurrence x et y (plus courts) ont une seule dérivation. Donc w aussi.

Question 32

Heap(1) affiche une fois : $A(1) = 1$. Pour $k \geqslant 2$, la boucle fait k appels à Heap$(k-1)$, et les échanges n'affichent rien : $A(k) = kA(k-1)$.

Récurrence : si $A(k-1) = (k-1)!$, alors $A(k) = k\,(k-1)! = k!$.

$$A(k) = k!.$$

Question 33

Initialisation : Heap(1) affiche sans modifier : 1 est impair, t est inchangé.

Hérédité : soit $k \geqslant 2$, avec $P(k-1)$ vraie. Notons $a_0, \dots, a_{k-1}$ les k premières cases au départ.

Cas k pair : $k-1$ est impair, donc chaque appel Heap$(k-1)$ laisse t inchangé. Il ne reste que les échanges de $t[i]$ et $t[k-1]$ pour $i = 0, \dots, k-1$:

- $i = 0$: $[a_{k-1}, a_1, \dots, a_{k-2}, a_0]$;
- $i = 1$: $[a_{k-1}, a_0, a_2, \dots, a_{k-2}, a_1]$;
- après l'étape i : $[a_{k-1}, a_0, \dots, a_{i-1}, a_{i+1}, \dots, a_{k-2}, a_i]$;
- $i = k-2$: $[a_{k-1}, a_0, \dots, a_{k-3}, a_{k-2}]$; $i = k-1$: échange de $t[k-1]$ avec lui-même.

Résultat : $[a_{k-1}, a_0, a_1, \dots, a_{k-2}]$, la rotation circulaire vers la droite annoncée.

Cas k impair ($k \geqslant 3$) : $k-1$ est pair, donc chaque appel fait tourner vers la droite les $k-1$ premières cases. Un tour de boucle (appel puis échange de $t[0]$ et $t[k-1]$) transforme $[c_0, \dots, c_{k-1}]$ en :

$$[c_0, \dots, c_{k-2}, c_{k-1}] \xrightarrow{\text{rotation des } k-1} [c_{k-2}, c_0, \dots, c_{k-3}, c_{k-1}] \xrightarrow{t[0] \leftrightarrow t[k-1]} [c_{k-1}, c_0, \dots, c_{k-3}, c_{k-2}],$$

c'est-à-dire une rotation vers la droite des k cases. La boucle fait k tours, donc k rotations de k cases : on revient au tableau de départ. Les cases au-delà de k ne sont jamais touchées.

$P(k)$ est vraie, ce qui achève la récurrence.

Question 34

Montrons par récurrence : « Heap(k, t) affiche une fois chacune des $k!$ permutations des k premières cases, les autres étant inchangées ».

$k = 1$: une seule permutation, affichée une fois.

Hérédité : au tour i, l'appel Heap$(k-1)$ laisse $t[k-1]$ fixe (il ne touche que les $k-1$ premières cases) et affiche, par hypothèse de récurrence, toutes les permutations des $k-1$ premières cases avec cette valeur fixe en position $k-1$. Il suffit de voir que les k valeurs présentes en position $k-1$ au début des k tours sont **distinctes** :

- k pair : d'après le calcul de Q33, avant le tour $i \geqslant 1$ la dernière case contient a_{i-1}, et avant le tour 0 elle contient a_{k-1} : on obtient $a_{k-1}, a_0, a_1, \dots, a_{k-2}$, toutes distinctes ;
- k impair : chaque tour fait une rotation des k cases, donc avant le tour i la dernière case contient a_{k-1-i} : $a_{k-1}, a_{k-2}, \dots, a_0$, toutes distinctes.

Les k tours affichent donc des permutations deux à deux différentes (leur dernière case diffère d'un tour à l'autre), $(k-1)!$ par tour : $k!$ permutations distinctes, c'est-à-dire toutes (Q32).

Question 35

```
let pivot (k : int) (i : int) (t : 'a array) : unit =
  let swap a b = let x = t.(a) in t.(a) <- t.(b); t.(b) <- x in
  if k mod 2 = 0 then swap i (k - 1)     (* k pair : t[i] <-> t[k-1] *)
  else swap 0 (k - 1)                    (* k impair : t[0] <-> t[k-1] *)
```

Question 36

Idée : on simule la récursion de Heap de façon paresseuse. `heap k next` renvoie le flot des tableaux affichés par Heap(k, t), suivi de `next ()`. Un affichage devient un `Cons` contenant une **copie** du tableau de travail ; les échanges ont lieu au moment où l'on demande la suite.

```
let permstream_of_array (t0 : 'a array) : 'a array stream =
  let t = Array.copy t0 in                 (* tableau de travail *)
  let n = Array.length t in
  let rec heap k (next : unit -> 'a array stream) : 'a array stream =
    if k <= 1 then Cons (Array.copy t, next)          (* « afficher » *)
    else
      let rec loop i () =                             (* boucle pour i = 0 .. k-1 *)
        if i = k then next ()
        else heap (k - 1) (fun () -> pivot k i t; loop (i + 1) ())
      in
      loop 0 ()
  in
  heap n (fun () -> Nil)
```

Mémoire : un seul tableau de travail et une chaîne de fermetures de profondeur au plus n : $O(n)$ en plus des tableaux effectivement demandés, au lieu de stocker les $n!$ permutations. On n'en calcule une que si on la demande.

Attention : le flot fait des effets de bord quand on avance ; il doit être parcouru une seule fois (appeler deux fois la même queue rejouerait les échanges).

Le piège

1. En Q14, croire que `map` ou `intertwin` est en cause : c'est l'appel récursif `product_wrong` calculé avant le `Cons` qui ne termine pas.
2. En Q21, utiliser le même nombre h pour choisir le sous-flot et calculer ϱ : les bits de poids faible seraient corrélés au numéro du sous-flot. D'où le quotient h/m.
3. En Q25, oublier que $\neg\bot$ se démontre (par $\neg_i$) : ce n'est pas une hypothèse disponible.
4. En Q33, oublier l'échange du dernier tour ($i = k-1$) : dans la variante étudiée, il y a un échange après chaque appel, y compris le dernier.

La méthode

1. Flot paresseux : renvoyer tout de suite un `Cons` dont la queue est une fonction ; ne jamais forcer une queue avant d'en avoir besoin.
2. Estimation probabiliste : un résumé de taille fixe (maximum de ϱ) et une moyenne (harmonique) sur plusieurs sous-flots pour réduire la variance.
3. Chiffrement par flot : message $\oplus$ clé ; on déchiffre en refaisant $\oplus$ clé, car $k \oplus k = F$ et $F \oplus c = c$.
4. Dérécursifier paresseusement : passer à chaque appel une fonction « suite » qui calcule le reste du flot, et ne faire les effets qu'au moment où l'on force la queue.

Rappels de cours utilisés

- **Flots paresseux.** `Cons (x, f)` : la queue n'est calculée que lorsqu'on appelle `f ()`. Le corps d'une fonction `fun () -> ...` n'est pas évalué à sa création : c'est ce qui permet des suites infinies sans boucle infinie.
- **Indécidabilité de l'arrêt.** Théorème de Turing : il n'existe pas de programme qui, pour tout programme P et toute entrée x, termine et répond correctement à « P termine-t-il sur x ? ».
- **Verrous (Mutex).** Une section critique accède à une donnée partagée entre fils d'exécution. On l'encadre par `Mutex.lock` et `Mutex.unlock` d'un même verrou : un seul fil à la fois peut l'exécuter.
- **Déduction naturelle (règles utilisées).** (ax) $\Delta, A \vdash A$; $(\wedge_e)$ de $A \wedge B$ on tire A (ou B) ; $(\wedge_i)$ de A et B on tire $A \wedge B$; $(\vee_i)$ de A on tire $A \vee B$; $(\vee_e)$ de $A \vee B$, $\Delta, A \vdash C$, $\Delta, B \vdash C$ on tire C ; $(\neg_i)$ de $\Delta, A \vdash \bot$ on tire $\neg A$; $(\neg_e)$ de A et $\neg A$ on tire $\bot$; $(\bot_e)$ de $\bot$ on tire tout.
- **Grammaire non ambiguë.** Une grammaire est non ambiguë si chaque mot du langage a un seul arbre de dérivation. Pour $S \to aSbS \mid \varepsilon$, le b qui ferme le premier a est déterminé : c'est la fin du plus court préfixe non vide ayant autant de a que de b.